
Python for Everybody

Course 3:

USING PYTHON TO ACCESS WEB DATA

MODULE 2:(REGULAR EXPRESSIONS)

- In computing ,a regular expression , also referred to as “regex” or “regexp”,provides a concise and flexible means for matching strings of text,such as particular characters,words,or patterns of characters.
- A regular expression is written in a formal language that can be interpreted by the regular expression processor.
- Regular expressions are powerful and cryptic at the same time.
- Python provides the **re module** for working with regular expressions.

1. *Basic functions:*

Function	Description	
<code>re.match(pattern, string)</code>	Checks for a match only at the beginning of the string	
<code>re.search(pattern, string)</code>	Searches for the first occurrence of the pattern anywhere	
<code>re.findall(pattern, string)</code>	Returns all occurrences of the pattern in a list	
<code>re.finditer(pattern, string)</code>	Returns an iterator of match objects	
<code>re.sub(pattern, repl, string)</code>	Replaces occurrences with <code>repl</code>	
<code>re.split(pattern, string)</code>	Splits string by the pattern	

2. Common patterns:

Pattern	Meaning
<code>.</code>	Any character except newline
<code>^</code>	Start of string
<code>\$</code>	End of string
<code>\d</code>	Digit (0-9)
<code>\D</code>	Non-digit
<code>\w</code>	Word character (letters, digits, underscore)
<code>\W</code>	Non-word character
<code>\s</code>	Whitespace
<code>\S</code>	Non-whitespace
<code>[abc]</code>	Matches a , b , or c
<code>[^abc]</code>	Matches anything except a, b, or c
<code>`a`</code>	b`
<code>(abc)</code>	Capturing group

3. Quantifiers:

Quantifier	Meaning
<code>*</code>	0 or more
<code>+</code>	1 or more
<code>?</code>	0 or 1
<code>{n}</code>	Exactly n times
<code>{n,}</code>	n or more times
<code>{n,m}</code>	Between n and m times

- *Example:*

```
hand = open('mbox-short.txt')
for line in hand:
    line = line.rstrip()
    if line.find('From:') >= 0:
        print(line)
```

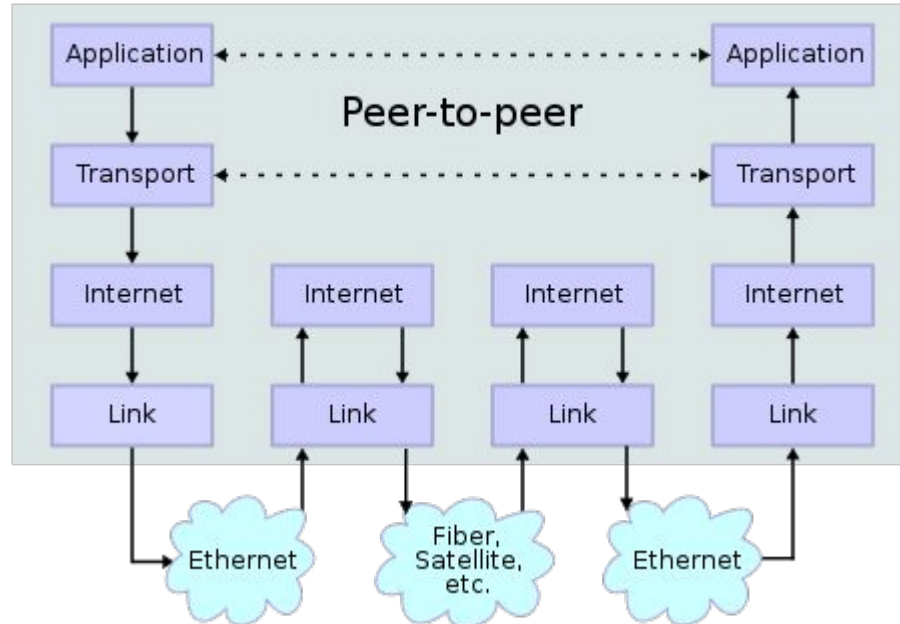
```
import re
```

```
hand = open('mbox-short.txt')
for line in hand:
    line = line.rstrip()
    if re.search('From:', line) :
        print(line)
```

MODULE 3:(NETWORK PROGRAMS)

- **TCP(transport control protocol):**
- Build on top of IP(internet protocol)
- Assumes IP might lose some data-stores and retransmits data if it seems to be lost.
- Handles “flow control” using a transmit window.
- It gives a nice pipeline.

Stack Connections



- **Port number:**
- A port number is an application-specific or process-specific software communications endpoint.
- It allows multiple networked applications to coexist on the same server.
- There is a list of well known port number used with different types applications:

Port	Protocol	Application	
20, 21	FTP	File Transfer Protocol (20 = data, 21 = control)	
22	SSH	Secure Shell (remote login, secure file transfer)	
23	Telnet	Remote login (unsecure, rarely used today)	
25	SMTP	Simple Mail Transfer Protocol (sending email)	
53	DNS	Domain Name System (name resolution)	
67, 68	DHCP	Dynamic Host Configuration Protocol (IP assignment)	
69	TFTP	Trivial File Transfer Protocol	
80	HTTP	HyperText Transfer Protocol (web traffic)	
110	POP3	Post Office Protocol (receiving email)	
119	NNTP	Network News Transfer Protocol	
123	NTP	Network Time Protocol (time sync)	
143	IMAP	Internet Message Access Protocol (email retrieval)	

143	IMAP	Internet Message Access Protocol (email retrieval)
161, 162	SNMP	Simple Network Management Protocol
194	IRC	Internet Relay Chat
443	HTTPS	Secure HTTP (encrypted web traffic)
465	SMTPS	Secure SMTP
514	Syslog	System logging
587	SMTP (submission)	Sending mail securely
636	LDAPS	Secure LDAP
3389	RDP	Remote Desktop Protocol
8080	HTTP-alt	Alternative web traffic port

- **Sockets:**

- A socket is an endpoint for communication between two machines over a network.
- In python, sockets module provides low-level networking interface.

Python has built-in support for TCP Sockets

```
import socket
mysock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
mysock.connect( ('data.pr4e.org', 80) )
```

Host

Port

- **Import socket:** this imports the socket module ,which provides access to berkeley socket interface.
- **mysock = socket.socket(socket.AF_INET,socket.SOCK_STREAM):**this line create an **socket object** . **socket.AF_INET** specifies the address family as IPV4 , and **socket.SOCK_STREAM** indicates a TCP socket(stream-oriented).
- **mysock.connect (...):** this establishes connection to a remote host at the specified port number(80).

- **Telnet:**

- A network protocol that allows you to .log into another computer over a tcp/ip network.
- Runs on port 23 by default.
- It provides a command-line interface to communicate with remote devices.
- Common uses:
 1. Testing connectivity to a specific port
 2. Accessing older network devices like router and switches.
- Why is it not used:
 1. Insecure- everything is displayed as normal plain text without encryption./
 2. Replaced by SSH,which encrypts communication and is the modern standard.

```
$ telnet data.pr4e.org 80
Trying 74.208.28.177...
Connected to data.pr4e.org.
Escape character is '^['.
GET http://data.pr4e.org/page1.htm HTTP/1.0

HTTP/1.1 200 OK
Date: Tue, 30 Jan 2024 15:30:13 GMT
Server: Apache/2.4.18 (Ubuntu)
Last-Modified: Mon, 15 May 2017 11:11:47 GMT
Content-Length: 128
Content-Type: text/html

<h1>The First Page</h1>
<p>If you like, you can switch to
the <a href="http://data.pr4e.org/page2.htm">Second
Page</a>.</p>
```

- **HTTP:**

- Http stands for *hypertext transfer protocol*.
- It is the foundation of communication on the world wide web.
- Http works on port 80 by default.
- It defines how the browsers and server exchanges information/messages.
- But for extracting data from a source first we need to feed some then only we can extract.

- **Getting data from server:**

1. When we click on a link from a webpage.
 2. The browser makes a connection to the web server and issues a “GET” request
 3. The get request is made to get the content of the page at the specified url.
 4. Then the server returns the HTML document to the browser which formats and displays the documents to user.
- The diagram is displayed in the next page.

Request

Web Server

Response

80

GET http://www.dr-chuck.com/page2.htm

<h1>The Second
Page</h1><p>If you like, you
can switch back to the First
Page</p>

Browser



Click

Parse/
Render



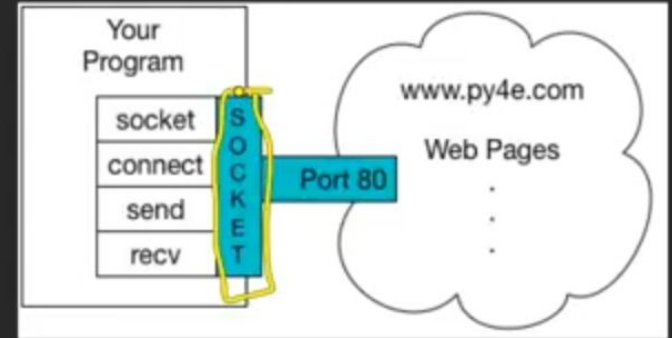
Request-Response Cycle

- An HTTP request in python:

```
import socket

mysock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
mysock.connect(('data.pr4e.org', 80))
cmd = 'GET http://data.pr4e.org/romeo.txt HTTP/1.0\r\n\r\n'.encode()
mysock.send(cmd)

while True:
    data = mysock.recv(512)
    if (len(data) < 1):
        break
    print(data.decode())
mysock.close()
```



- In HTTP we need to send data first then only we can receive
- First we send the data from our to the connected server(line4)
- Then we start retrieving the entire data(while loop).

- Python script to retrieve document from a web server via http.

```
socket1.py
1  import socket
2
3  mysock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
4  mysock.connect(('data.pr4e.org', 80))
5  cmd = 'GET http://data.pr4e.org/romeo.txt HTTP/1.0\n\n'.encode()
6  mysock.send(cmd)
7
8  while True:
9      data = mysock.recv(512)
10     if (len(data) < 1):
11         break
12     print(data.decode())
13 mysock.close()
14
```

MODULE 4:(PROGRAMS THAT SURF THE WEB)

- Character set:
- ASCII(AMERICAN STANDARD CODE FOR INFORMATION INTERCHANGE)
- It is the standard encoding character set that maps the characters with numbers.
- Cheat sheet link:

● <https://www.ascii-code.com/>

- **Representing simple string:(ASCII-older-character set):**
- Each character is represented by a number between 0 to 256 stored in memory.
- The ord() function tell us the numeric value of a simple ASCII character.

```
>>> print(ord('H'))  
72  
>>> print(ord('e'))  
101  
>>> print(ord('\n'))  
10
```

- **Unicode:**
- Unicode is universal character encoding standard.
- Goal to use unicode: every character from every language(letters,symbols,emoji,script) in a consistent way across computers
- It assigns a unique code point to each character.
- To represent the wide range of characters computers must handle we represent characters with more than one byte:

>> UTF-16: fixed length-two bytes
>> UTF-32: fixed length-four bytes
>> UTF-8-1-4 bytes(variable length)
>>> Upwards compatible with ASCII.
>>> Automatic detection between ASCII & UTF-8
>>> UTF-8 is recommended practice for encoding data to be exchanged between systems

- Major change between python2 and python3 was unicode
- In python2 we were supposed to specify whether the data is unicode
- But in python3 all strings are unicode.

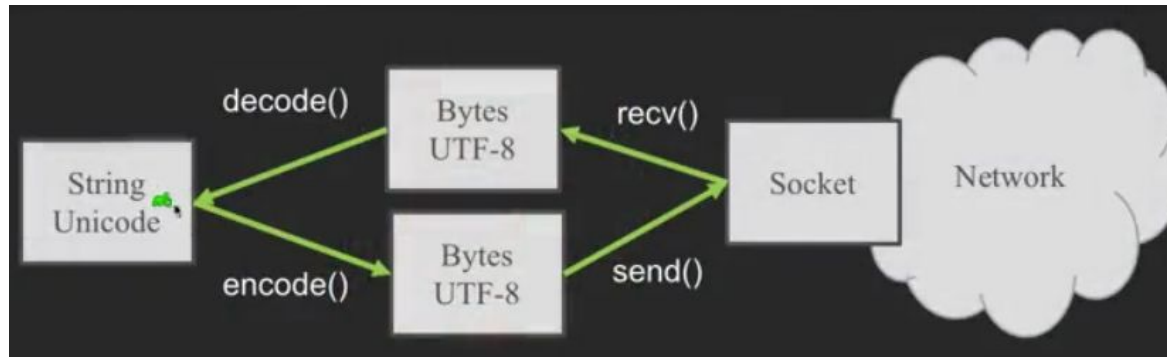
Python 2.7.10

```
>>> x = '이광춘'
>>> type(x)
<type 'str'>
>>> x = u'이광춘'
>>> type(x)
<type 'unicode'>
>>>
```

Python 3.5.1

```
>>> x = '이광춘'
>>> type(x)
<class 'str'>
>>> x = u'이광춘'
>>> type(x)
<class 'str'>
>>>
```


- In python 3 ,all strings internally are unicode
- When we have to talk o a database we have to encode and decode data(usually to UTF-8).
- When we talk to external resource like a network socket we sends byte ,so we need to encode Python 3 strings into a given character encoding.
- When we read data from an external resource ,we must decode it based on the character set so it properly represented in python 3 as string.
- After decoding the data is either ASCII OR UTF-8:
 1. Old data:ASCII
 2. New data:UTF-8
- **decode()**: returns a string from a given byte
- **encode()**: returns an encoded version of string as bytes
-



- **Urllib:**

- This is a library that wraps the socket connection and does it automatically.
- Instead of writing ten lines of code for socket connection we can directly use urllib.
- But urllib returns file data.

```
import urllib.request, urllib.parse, urllib.error
```

```
fhand = urllib.request.urlopen('http://data.pr4e.org/romeo.txt')  
for line in fhand:  
    print(line.decode().strip())
```

- **Web scraping:**

- **What is web scraping?**

- When a program or script pretends to be a browser and retrieves web pages, looks at those pages, extracts information, and then looks at more web pages.
- Search engine scrapes web pages—we call it as “spidering the web or web crawling”.

- **Why web scraping?**
- Pull data-particularly social data
- Get back your own data back out of some system that has no export capabilities
- Monitor site for new information.
- Spider the web to make a database for a search engine.
- **Scraping web pages:**
- Republishing copyrighted information is not allowed.
- Violating terms of service is not allowed.
- **Beautiful soup:**
- A library used for web scraping-extracting data from html and xml documents.
- This create a parse tree that makes it easy to navigate,search,and modify data in web pages.
- And this is also used for handling poorly formatted HTML gracefully.
- To use beautiful soup then firstly we need install beautiful soup

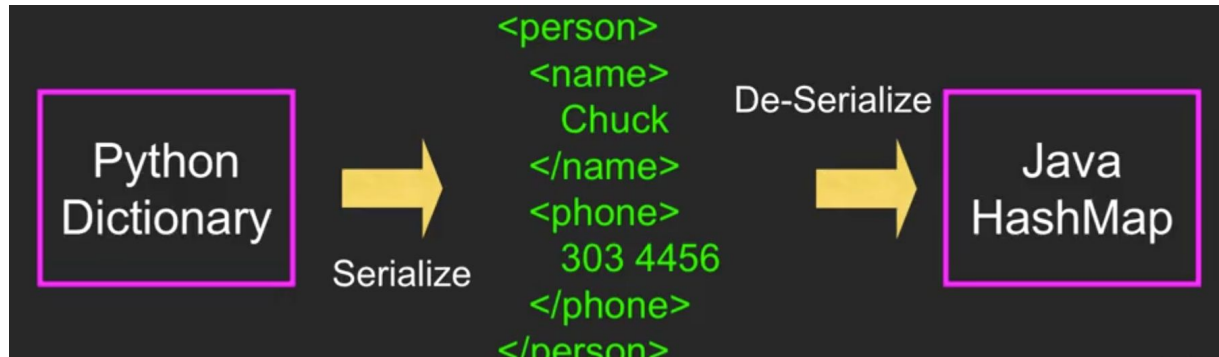
```
import urllib.request, urllib.parse, urllib.error  
from bs4 import BeautifulSoup
```

```
url = input('Enter - ')  
html = urllib.request.urlopen(url).read()  
soup = BeautifulSoup(html, 'html.parser')
```

```
# Retrieve all of the anchor tags  
tags = soup('a')  
for tag in tags:  
    print(tag.get('href', None))
```

MODULE 5:(WEB SERVICES AND XML)

- Data on the web:
- The format in which Data to be transferred can be done in any language that we agree on .this a topic of argument.
- Instead we can use wire format:a standardized way of representing data for transmission between systems,enabling interoperability
- It uses two method:
 1. Serialization:the process of converting a python dictionary into a common “wire format”.(like XML OR JSON)
 2. De-serialization: the process of converting the “wire format” back into a language-specific data structure,such as java hashmap.



- **XML:(extensible markup language):**
- It is a markup language like HTML, but designed to store and transport data, not to display it.
- Key features:
 1. Self-descriptive: data is stored in a tree-like structure.
 2. Custom-tags: unlike HTML we can define our own tags.
 3. Hierarchical: data is stored in tree-like structure
 4. Platform-independent: works across system languages and devices.
 5. Used for data exchange: common in API'S ,configuration files ,web services,etc.
- It started as simplified subset of the standard generalized markup language, and is designed to be relatively human-legible

```
<people>
  <person>
    <name>Chuck</name>
    <phone>303 4456</phone>
  </person>
  <person>
    <name>Noah</name>
    <phone>622 7421</phone>
  </person>
</people>
```

- **XML SCHEMA:**
- An xml schema defines the structure,content,and data types of XML documents .
- It is like a blueprint or contract that tells what elements and attributes an XML document can contain and in what order.
- Contract itself a xml.
- It's is often referred as schema because schema is generic but commonly it is called XSD because the file names end in .xsd

Many XML Schema Languages

- Document Type Definition (DTD)
 - http://en.wikipedia.org/wiki/Document_Type_Definition
- Standard Generalized Markup Language (ISO 8879:1986 SGML)
 - <http://en.wikipedia.org/wiki/SGML>
- XML Schema from W3C - (XSD)
 - [http://en.wikipedia.org/wiki/XML_Schema_\(W3C\)](http://en.wikipedia.org/wiki/XML_Schema_(W3C))



- **XSD data types:**

```
<xs:element name="customer" type="xs:string"/>
<xs:element name="start" type="xs:date"/>
<xs:element name="startdate" type="xs:dateTime"/>
<xs:element name="prize" type="xs:decimal"/>
<xs:element name="weeks" type="xs:integer"/>
```

XSD Data Types

It is common to represent time in UTC/GMT, given that servers are often scattered around the world.

```
<customer>John Smith</customer>
<start>2002-09-24</start>
<startdate>2002-05-30T09:30:10Z</startdate>
<prize>999.50</prize>
<weeks>30</weeks>
```

- **Parsing XML:**

- It takes a XML tags as string and returns a tree
- Then extracting data from the tree by using different functions.
- In the last line get method is used for getting the value of attribute hide

```
import xml.etree.ElementTree as ET
data = '''<person>
  <name>Chuck</name>
  <phone type="intl">
    +1 734 303 4456
  </phone>
  <email hide="yes"/>
</person>'''

tree = ET.fromstring(data)
print('Name:', tree.find('name').text)
print('Attr:', tree.find('email').get('hide'))
```


- **JSON(JAVASCRIPT OBJECT NOTATION):**
- Json is simpler than xml
- Json is a lightweight data format used for storing and exchanging data.
- It's human-readable and language-independent but it is derived from javascript.
- Widely used in APIs and config files.
- In json data is represented in key:value format,that is dictionary format.

```
import json
data = '''{
    "name" : "Chuck",
    "phone" : {
        "type" : "intl",
        "number" : "+1 734 303 4456"
    },
    "email" : {
        "hide" : "yes"
    }
}'''

info = json.loads(data)
print('Name:',info["name"])
print('Hide:',info["email"]["hide"])
```

- **Service oriented approach:**

- Most non-trivial web applications uses services.
- They use services from other applications
 1. Credit card charge
 2. Hotel reservation systems
- It makes use of API to connect with other application.
- Services oriented approach is a way how 2 applications share data.

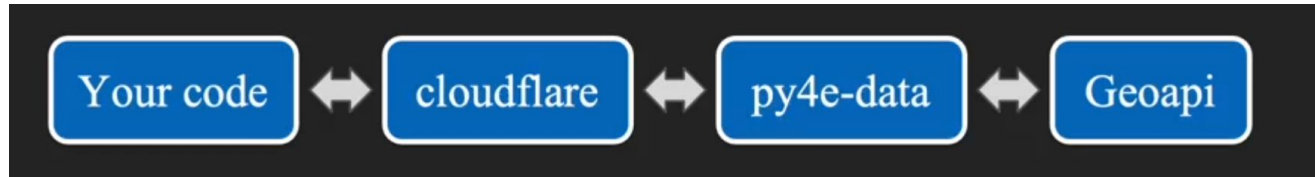
- **Using API:**

-

There Are Many APIs

- There are organizations that put up public APIs and sell access to those APIs
- We will explore a geocoding API based on the OpenStreetMap data
- You need an account to access this API
- There is a free level of requests
- You pay above that rate of usage

- **Geoapify :**
- This is a location-based service(lbs) platform provides powerful APIs for working with maps,geocoding, and geographic data.
- Developers often use it to add location intelligence to their app
- **Api proxy:**
- An api proxy server sits between your client and the target api.
- It's like a middleman
- In this flow without creating an account on geo apify we can still geoapify through proxy.



```
import urllib.request, urllib.parse
import http, json, ssl

serviceurl = 'https://py4e-data.dr-chuck.net/opengeo?'

while True:
    address = input('Enter location: ')
    if len(address) < 1: break

    address = address.strip()
    parms = dict()
    parms['q'] = address

    url = serviceurl + urllib.parse.urlencode(parms)

    print('Retrieving', url)
    uh = urllib.request.urlopen(url, context=ctx)
    data = uh.read().decode()
    print('Retrieved', len(data), 'characters', data[:20].replace('\n', ' '))

    js = json.loads(data)

    lat = js['features'][0]['properties']['lat']
    lon = js['features'][0]['properties']['lon']
    print('lat', lat, 'lon', lon)
    location = js['features'][0]['properties']['formatted']
```

Enter location: Ann Arbor, MI
Retrieving https://py4e-data.
dr-chuck.net/opengeo?q=Ann+Arbor%2C+MI
Retrieved 1319 characters {"type":"FeatureColl
lat 42.2681569 lon -83.7312291
Ann Arbor, MI, United States of America

opengeo.py